# Greedy Routing for Post-Disaster Emergency Response in Cagayan de Oro City

A Traveling Salesperson Problem solved with Algorithm 2.7 (Greedy / Nearest-Neighbor) on the real OpenStreetMap road network

Jhey Gulde

University of Science and Technology of Southern Philippines

Performance Innovative Task – Algorithm Discussion and Real-Life Application of the TSP

October 2026

**Abstract**

After a typhoon, an emergency response crew in Cagayan de Oro City must leave a central depot, reach ten critical sites (hospitals, evacuation centers and a response hub) exactly once, and return, in the shortest possible time. We model this as a directed, weighted Traveling Salesperson Problem (TSP). Site coordinates come from OpenStreetMap; travel times are fastest paths on the city's real drivable road network (5,835 intersections), and a road-clearing penalty models flood damage. We apply Algorithm 2.7 of Luna-Garcia, Candano and Caga-anan (2024), the Greedy (Nearest-Neighbor) approach, and implement it in Python. Starting from City Hall, the greedy tour takes **125.9 minutes** and is verified to visit every site exactly once. An exact Held–Karp dynamic program, confirmed by brute force over all $9! = 362{,}880$ tours, gives an optimum of **117.0 minutes**: the greedy tour is 7.6% longer. Because both tours spend the same 75 minutes clearing roads, the whole gap comes from driving. The result illustrates the algorithm's main trade-off: a fast, valid, easily explained route that is approximate rather than optimal.

## Contents

**1 Introduction** **3**

**2 The assigned algorithm: Algorithm 2.7 (Greedy)** **3**
- 2.1 Idea . . . 3
- 2.2 Procedure . . . 3
- 2.3 Time complexity . . . 4
- 2.4 Advantages . . . 4
- 2.5 Limitations . . . 5
- 2.6 Optimality . . . 5

**3 Problem and objective** **5**
- 3.1 Real-life situation . . . 5
- 3.2 Locations . . . 5
- 3.3 Assumptions . . . 5
- 3.4 Constraints . . . 6
- 3.5 Objective . . . 6

**4 Methodology and graph model** **6**
4.1 Weighted graph . . . 6
4.2 Data pipeline . . . 7
4.3 Clearing-time model . . . 8
4.4 Weight matrices . . . 8
4.5 Actual versus assumed data . . . 9

**5 Applying Algorithm 2.7** **9**

**6 Python implementation** **11**
6.1 Structure . . . 11
6.2 Main components . . . 12
6.3 Program output . . . 13

**7 Results** **13**
7.1 Greedy route . . . 13
7.2 Verification . . . 14
7.3 Comparison with the optimum . . . 14
7.4 Sensitivity to the starting vertex . . . 16

**8 Conclusion** **16**
8.1 Interpretation . . . 16
8.2 Limitations . . . 16
8.3 Possible improvements . . . 16

**A Source code** **17**
A.1 `tsp_greedy_cdo.py` . . . 17
A.2 `build_osm_dataset.py` . . . 20
A.3 `tsp_graph_visualization.py` . . . 22

**B Road distances** **26**

# 1 Introduction

The Traveling Salesperson Problem (TSP) asks for a route that visits each of a given set of locations exactly once and returns to the starting location while minimizing the total distance, travel time or cost. It is one of the most studied problems in combinatorial optimization and is NP-hard: the number of possible tours grows factorially with the number of locations, so exact methods quickly become impractical [1].

This report documents a complete application of the TSP to a real situation in Cagayan de Oro City. It follows the structure of the Performance Innovative Task:

1. a discussion of the assigned algorithm, Algorithm 2.7 (Greedy), including its procedure, complexity, advantages, limitations and optimality (Section 2);
2. the real-life problem, its locations, assumptions, constraints and objective (Section 3);
3. the weighted graph model and the data behind it, with actual and assumed data clearly separated (Section 4);
4. the application of the algorithm with all key calculations (Section 5);
5. the Python implementation (Section 6);
6. the results, their verification and an optimality check (Section 7); and
7. conclusions, limitations and possible improvements (Section 8).

**Summary of results.** The greedy round trip from City Hall is $0 \to 1 \to 9 \to 4 \to 3 \to 8 \to 7 \to 2 \to 6 \to 5 \to 0$ with a total of 125.9 minutes (50.9 minutes driving, 75 minutes clearing, 27.03 km). The exact optimum is 117.0 minutes, so the greedy solution is 8.9 minutes (7.6%) longer.

# 2 The assigned algorithm: Algorithm 2.7 (Greedy)

## 2.1 Idea

A greedy algorithm builds a solution step by step and, at every step, takes the choice that looks best *right now*, without considering its long-term consequences [1]. For the TSP this choice is simple: from the current location, travel to the *nearest location that has not been visited yet*. For this reason the method is also called the Nearest-Neighbor heuristic. It never backtracks or revises an earlier choice.

## 2.2 Procedure

Algorithm 1 reproduces the pseudocode of Algorithm 2.7 from the paper. Algorithm 2 states the same procedure precisely, in the form that our Python code implements, for a weight matrix $W$ and a starting vertex $s$.

**Algorithm 1** Algorithm 2.7 – Greedy Algorithm Approach for TSP (Luna-Garcia et al., 2024, p. 89)

**Input:** cities and pairwise distances
Begin at an arbitrary city
**while** not all cities have been visited **do**
    Find the nearest unvisited city (based on the distance) and travel to it
    Mark the city as visited
**end while**
Return to the starting city to complete the tour
**Output:** complete tour with the total distance

**Algorithm 2** Greedy TSP, formal version (as implemented in `greedy_tsp`)

**Require:** weight matrix $W \in \mathbb{R}^{n \times n}$, start vertex $s$

$tour \leftarrow [s]$; $visited \leftarrow \{s\}$; $total \leftarrow 0$; $u \leftarrow s$
**while** $|tour| < n$ **do**
  $v \leftarrow \arg\min_{j \notin visited} W[u][j]$ ▷ nearest unvisited vertex
  append $v$ to $tour$; $visited \leftarrow visited \cup \{v\}$
  $total \leftarrow total + W[u][v]$; $u \leftarrow v$
**end while**
$total \leftarrow total + W[u][s]$; append $s$ to $tour$ ▷ close the loop
**return** $tour, total$

Step by step:

1. **Initialize.** Put the starting vertex in the tour and mark it visited.
2. **Look around.** From the current vertex $u$, read the weights $W[u][j]$ to every unvisited vertex $j$.
3. **Choose greedily.** Pick the unvisited vertex $v$ with the smallest weight. Ties are broken by the lowest index.
4. **Move.** Add $v$ to the tour, mark it visited, add $W[u][v]$ to the total and make $v$ the current vertex.
5. **Repeat** steps 2–4 until every vertex is in the tour.
6. **Close the tour** by returning from the last vertex to the start and adding that weight.

Because every vertex is added exactly once and the tour ends at the start, the output is always a valid closed tour (a Hamiltonian cycle).

### 2.3 Time complexity

**Theorem 1** (Theorem 2.7 in [1])**.** *For $n$ cities, the greedy approach for the TSP runs in $O(n^2)$ time.*

*Proof.* Choosing the start and initializing the arrays costs $O(n)$. The loop runs $n-1$ times. In each iteration, finding the nearest unvisited vertex scans at most $n$ entries of one row of $W$, which is $O(n)$; marking the vertex and updating the total are $O(1)$. The loop therefore costs $(n-1) \cdot O(n) = O(n^2)$. Closing the tour is $O(1)$. The total is $T(n) = O(n) + O(n^2) + O(1) = O(n^2)$. □

For our $n = 10$ sites this is at most $9 \times 10 = 90$ comparisons. The algorithm needs $O(n^2)$ memory for the weight matrix and $O(n)$ for the tour. For comparison, the exact methods in the same paper cost $O(n^2 2^n)$ (dynamic programming) and $O(n!)$ (brute force).

### 2.4 Advantages

- **Fast and scalable.** $O(n^2)$ time handles thousands of locations; the paper's experiments run it on up to 256 cities.
- **Simple.** It needs only a distance matrix and a visited list, and it is easy to explain to a field team.
- **Always valid.** It always returns a closed tour that visits every location exactly once.
- **Deterministic.** For a fixed start (and tie-breaking rule) it always gives the same tour, so results are reproducible.
- **A good starting point.** Its tour is a common initial solution for improvement methods such as 2-opt [7].

### 2.5 Limitations

- **Myopic.** Taking the shortest edge now can force very long edges later, especially the last legs and the return to the start.
- **Sensitive to the starting vertex.** Different starts give different tours and totals (Table 10).
- **No quality guarantee in general.** For metric instances its tour can be up to $\frac{1}{2}(\lceil \log_2 n \rceil + 1)$ times the optimum [6], and without the triangle inequality it can be arbitrarily bad.
- **No backtracking.** Unlike simulated annealing, genetic algorithms or particle swarm optimization [1], it cannot undo a poor early choice.

### 2.6 Optimality

**The greedy approach does not guarantee an optimal solution; it produces an approximate (heuristic) solution.** Each step is locally optimal, but the combined tour is generally not globally optimal. In this project we therefore claim optimality only for the exact Held–Karp dynamic program used as a check in Section 7.3, never for the greedy tour.

## 3 Problem and objective

### 3.1 Real-life situation

Cagayan de Oro City lies on the Cagayan River and has suffered severe river flooding, most notably during Tropical Storm Sendong (Washi) in December 2011. We consider a similar scenario: a typhoon has flooded and blocked roads across the city. A single emergency response crew leaves a central depot to deliver supplies and assess damage at every critical site, and must then return to the depot. Every minute on the road delays help at the remaining sites, so the order of visits matters.

### 3.2 Locations

The starting location (depot) is Cagayan de Oro City Hall. The nine required locations are four hospitals, four evacuation sites and one emergency response hub (Table 1). All coordinates were looked up on OpenStreetMap. City Hall is used as the depot because the only disaster-office entry in OpenStreetMap is in the provincial Capitol area and may be the provincial office. Schools are used as evacuation sites because OpenStreetMap does not tag evacuation centers explicitly.

**Table 1:** The ten vertices: real coordinates (OpenStreetMap), assumed damage class, and distance to the nearest road intersection.

| # | Site | Category | Latitude | Longitude | $d_i$ (min) | Snap (m) |
|---|---|---|---|---|---|---|
| 0 | Cagayan de Oro City Hall | Depot | 8.47695 | 124.64117 | 0 | 89 |
| 1 | Northern Mindanao Medical Center | Hospital | 8.48539 | 124.64964 | 0 | 81 |
| 2 | J. R. Borja Memorial City Hospital | Hospital | 8.48171 | 124.62809 | 5 | 82 |
| 3 | Capitol University Hospital | Hospital | 8.47905 | 124.67268 | 0 | 87 |
| 4 | Maria Reyna–Xavier University Hospital | Hospital | 8.47631 | 124.65520 | 0 | 37 |
| 5 | Balulang Elementary School | Evacuation | 8.44344 | 124.63451 | 20 | 77 |
| 6 | Macasandig Multipurpose Covered Court | Evacuation | 8.46535 | 124.64478 | 20 | 37 |
| 7 | Carmen National High School | Evacuation | 8.46950 | 124.63234 | 5 | 45 |
| 8 | West City Central School | Evacuation | 8.47350 | 124.63686 | 5 | 59 |
| 9 | Philippine Red Cross – CDO Chapter | Response hub | 8.48477 | 124.64926 | 0 | 127 |

### 3.3 Assumptions

1. **Driving speeds** depend on road class (Table 2): 20–40 km/h, typical of urban traffic. OpenStreetMap `maxspeed` tags are used where present.

2. **Road clearing** adds a delay at each site's access roads according to an assumed damage class: 0 (clear), 5 (minor debris) or 20 minutes (major obstruction).
3. **Riverside sites are hardest hit.** Balulang and Macasandig, along the Cagayan River, are assigned major obstruction; J. R. Borja, Carmen and West City, minor debris; the others are clear.
4. **Schools serve as evacuation centers.**
5. **Service time** at each site is not modeled, and the short walk from a building to the nearest road intersection (at most 127 m) is ignored.
6. **The network is static**: travel and clearing times do not change during the trip.

### 3.4 Constraints

1. The tour starts and ends at the depot (vertex 0).
2. Each of the other nine sites is visited exactly once.
3. One crew with one vehicle travels on drivable roads only.
4. Road direction is respected: one-way streets may only be driven in their allowed direction.
5. There are no time windows, capacity limits or site priorities.

### 3.5 Objective

Minimize the **total round-trip time in minutes**. For a tour $\pi = (\pi_0, \pi_1, \ldots, \pi_{n-1}, \pi_n)$ with $\pi_0 = \pi_n = 0$ that visits every site once,

$$\min_{\pi} \; T(\pi) = \sum_{k=0}^{n-1} w(\pi_k, \pi_{k+1}), \qquad w(i,j) = t(i,j) + c(i,j), \tag{1}$$

where $t(i,j)$ is the driving time and $c(i,j)$ is the road-clearing time from site $i$ to site $j$.

## 4 Methodology and graph model

### 4.1 Weighted graph

The problem is modeled as a complete weighted directed graph $G = (V, E, w)$.

- **Vertices** $V = \{0, 1, \ldots, 9\}$: the ten sites of Table 1; vertex 0 is the depot.
- **Edges** $E = \{(i,j) : i \neq j\}$: the fastest road route from site $i$ to site $j$ on the real network, giving $10 \times 9 = 90$ directed edges.
- **Direction.** Travel is *directed*, because one-way streets make the fastest route from $i$ to $j$ differ from the route from $j$ to $i$. In our data, 20 of the 45 site pairs differ by direction, by at most 0.6 minutes.
- **Weights** $w(i,j) = t(i,j) + c(i,j)$, in minutes.

Figure 1 shows the labeled graph with vertices at their real relative positions.

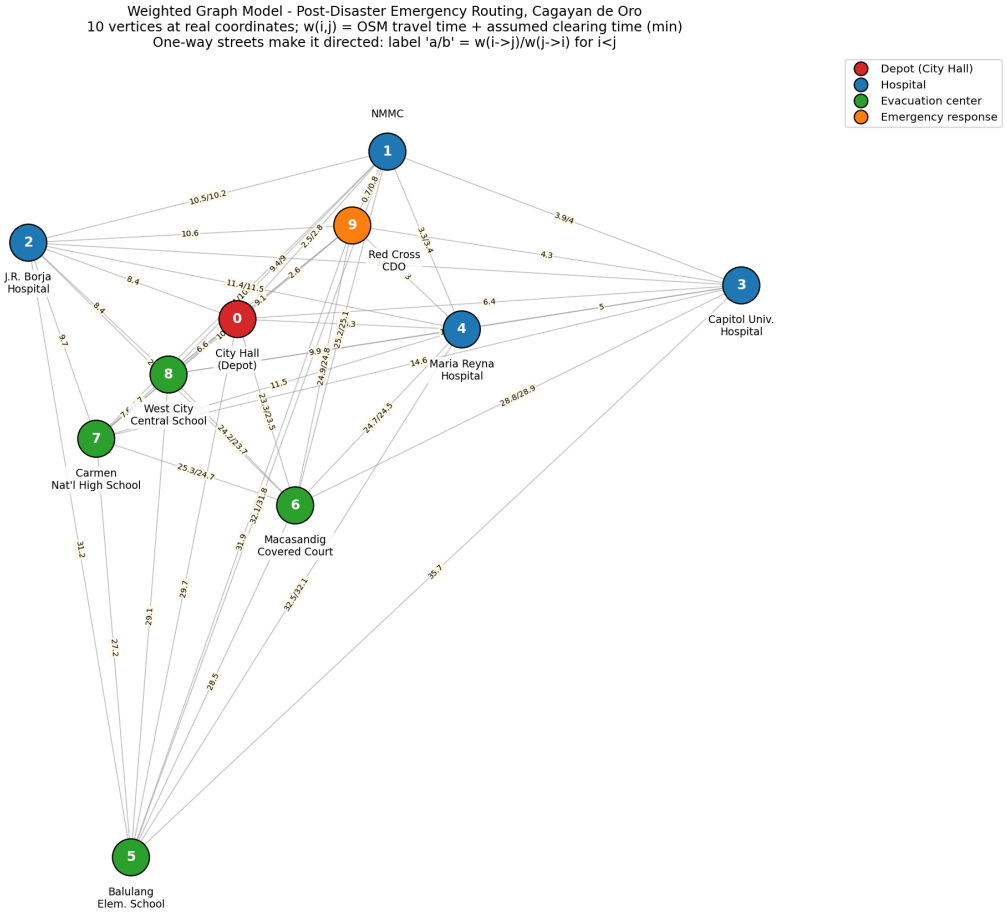



**Figure 1:** Weighted graph model. Vertex colors give the category; an edge label $a/b$ means $w(i \to j)/w(j \to i)$ for $i < j$, and a single number means both directions are equal.

## 4.2 Data pipeline

The script `build_osm_dataset.py` builds the actual travel-time data in five steps.

1. **Geocoding.** Each site was located with the OpenStreetMap Nominatim geocoder, and with Overpass API searches for specific buildings where the geocoder returned only a barangay center.
2. **Road network.** OSMnx [3] downloads the drivable network inside the sites' bounding box plus a 0.02° (about 2.2 km) margin. Only the largest strongly connected component is kept, so every site can reach every other. The result has 5,835 intersections and 14,346 directed road segments, and is saved as `data/cdo_drive.graphml`.
3. **Edge travel times.** Each road segment's speed comes from its `maxspeed` tag if present, otherwise from Table 2; its travel time is length divided by speed.
4. **Snapping.** Each site is attached to its nearest road intersection by great-circle (haversine) distance; all snaps are within 127 m (Table 1).
5. **Shortest paths.** Dijkstra's algorithm [4], run once from each site, gives the fastest driving time to every other site. Times are rounded to 0.1 minute. The road length along each fastest path is also recorded (Table 11).

**Table 2:** Assumed urban driving speeds by OpenStreetMap road class (km/h); other classes use 20 km/h.

| Road class | km/h | Road class | km/h | Road class | km/h |
|---|---|---|---|---|---|
| motorway | 60 | primary | 35 | tertiary | 25 |
| motorway_link | 40 | primary_link | 25 | tertiary_link | 20 |
| trunk | 40 | secondary | 30 | unclassified, residential | 20 |
| trunk_link | 30 | secondary_link | 25 | service / living_street | 15 / 10 |

### 4.3 Clearing-time model

Each site $i$ has an assumed damage class $d_i \in \{0, 5, 20\}$ minutes (Table 1). Reaching or leaving a damaged site requires clearing its access roads, so an edge costs the worse of its two endpoints:

$$c(i, j) = \max(d_i, d_j), \qquad c(i, i) = 0. \tag{2}$$

### 4.4 Weight matrices

Tables 3–5 give the three matrices, and Figure 2 shows them as heat maps. Rows and columns 5 and 6 (Balulang and Macasandig) stand out because of their 20-minute clearing penalty.

**Table 3:** Actual travel-time matrix $t(i, j)$ in minutes: fastest path on the OpenStreetMap drive network (row = from, column = to).

| from \ to | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | – | 2.5 | 3.4 | 6.4 | 3.3 | 9.7 | 3.3 | 3.3 | 1.6 | 2.6 |
| **1** | 2.8 | – | 5.5 | 3.9 | 3.3 | 12.1 | 5.2 | 6.0 | 4.4 | 0.7 |
| **2** | 3.4 | 5.2 | – | 9.1 | 6.4 | 11.2 | 6.1 | 4.7 | 3.4 | 5.6 |
| **3** | 6.4 | 4.0 | 9.1 | – | 5.0 | 15.7 | 8.8 | 9.6 | 8.0 | 4.3 |
| **4** | 3.3 | 3.4 | 6.5 | 5.0 | – | 12.5 | 4.7 | 6.5 | 4.9 | 3.0 |
| **5** | 9.7 | 11.8 | 11.2 | 15.7 | 12.1 | – | 8.5 | 7.2 | 9.1 | 11.9 |
| **6** | 3.5 | 5.1 | 6.5 | 8.9 | 4.5 | 8.5 | – | 5.3 | 4.2 | 4.9 |
| **7** | 3.3 | 5.6 | 4.7 | 9.6 | 6.5 | 7.2 | 4.7 | – | 2.6 | 5.8 |
| **8** | 1.6 | 4.0 | 3.4 | 7.9 | 4.9 | 9.1 | 3.7 | 2.7 | – | 4.1 |
| **9** | 2.6 | 0.8 | 5.6 | 4.3 | 3.0 | 11.9 | 4.8 | 5.8 | 4.1 | – |

**Table 4:** Assumed clearing-time matrix $c(i, j) = \max(d_i, d_j)$ in minutes.

| from \ to | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | – | 0 | 5 | 0 | 0 | 20 | 20 | 5 | 5 | 0 |
| **1** | 0 | – | 5 | 0 | 0 | 20 | 20 | 5 | 5 | 0 |
| **2** | 5 | 5 | – | 5 | 5 | 20 | 20 | 5 | 5 | 5 |
| **3** | 0 | 0 | 5 | – | 0 | 20 | 20 | 5 | 5 | 0 |
| **4** | 0 | 0 | 5 | 0 | – | 20 | 20 | 5 | 5 | 0 |
| **5** | 20 | 20 | 20 | 20 | 20 | – | 20 | 20 | 20 | 20 |
| **6** | 20 | 20 | 20 | 20 | 20 | 20 | – | 20 | 20 | 20 |
| **7** | 5 | 5 | 5 | 5 | 5 | 20 | 20 | – | 5 | 5 |
| **8** | 5 | 5 | 5 | 5 | 5 | 20 | 20 | 5 | – | 5 |
| **9** | 0 | 0 | 5 | 0 | 0 | 20 | 20 | 5 | 5 | – |

**Table 5:** Combined weight matrix $w(i,j) = t(i,j) + c(i,j)$ in minutes, used by Algorithm 2.7.

| from \ to | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | – | 2.5 | 8.4 | 6.4 | 3.3 | 29.7 | 23.3 | 8.3 | 6.6 | 2.6 |
| **1** | 2.8 | – | 10.5 | 3.9 | 3.3 | 32.1 | 25.2 | 11.0 | 9.4 | 0.7 |
| **2** | 8.4 | 10.2 | – | 14.1 | 11.4 | 31.2 | 26.1 | 9.7 | 8.4 | 10.6 |
| **3** | 6.4 | 4.0 | 14.1 | – | 5.0 | 35.7 | 28.8 | 14.6 | 13.0 | 4.3 |
| **4** | 3.3 | 3.4 | 11.5 | 5.0 | – | 32.5 | 24.7 | 11.5 | 9.9 | 3.0 |
| **5** | 29.7 | 31.8 | 31.2 | 35.7 | 32.1 | – | 28.5 | 27.2 | 29.1 | 31.9 |
| **6** | 23.5 | 25.1 | 26.5 | 28.9 | 24.5 | 28.5 | – | 25.3 | 24.2 | 24.9 |
| **7** | 8.3 | 10.6 | 9.7 | 14.6 | 11.5 | 27.2 | 24.7 | – | 7.6 | 10.8 |
| **8** | 6.6 | 9.0 | 8.4 | 12.9 | 9.9 | 29.1 | 23.7 | 7.7 | – | 9.1 |
| **9** | 2.6 | 0.8 | 10.6 | 4.3 | 3.0 | 31.9 | 24.8 | 10.8 | 9.1 | – |

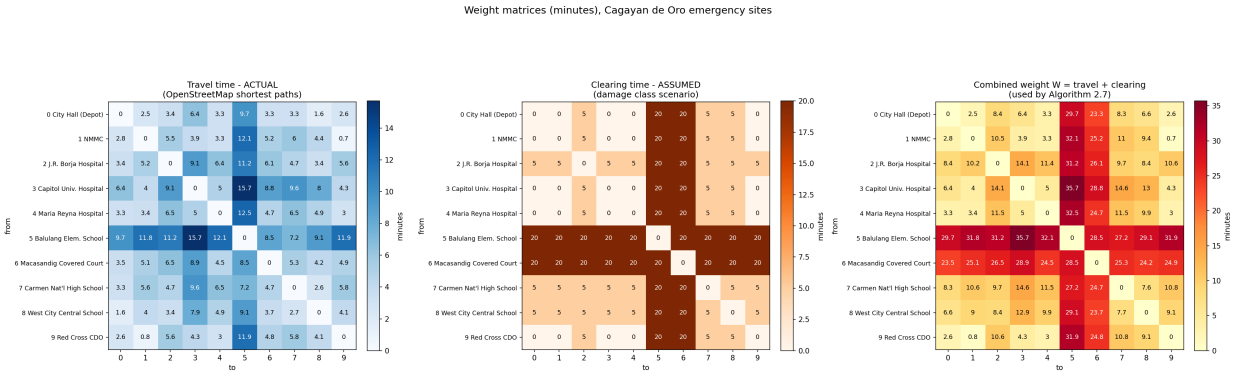



**Figure 2:** Travel time (actual), clearing time (assumed) and the combined weight matrix used by the algorithm.

### 4.5 Actual versus assumed data

**Table 6:** Data sources.

| Data | Source | Type |
|---|---|---|
| Site coordinates | OpenStreetMap via Nominatim and the Overpass API | Actual |
| Road network and road lengths | OpenStreetMap via OSMnx (5,835 intersections) | Actual |
| Route between sites | Dijkstra fastest path on the network | Actual |
| Driving speeds | Road-class speeds, Table 2 | Assumed |
| Clearing times | Damage classes 0 / 5 / 20 min, $c(i,j) = \max(d_i, d_j)$ | Assumed |
| Use of schools as evacuation centers | Scenario choice | Assumed |

OpenStreetMap data was accessed on 17 October 2026. Map data © OpenStreetMap contributors, available under the Open Database License [2].

## 5 Applying Algorithm 2.7

We run Algorithm 2.7 on the weight matrix of Table 5 with the depot, vertex 0, as the start. Table 7 traces every step.

**Table 7:** Trace of Algorithm 2.7 from the depot. At each step the three nearest unvisited sites are listed as *site:* $w$; the smallest is chosen.

| Step | Leg | Nearest unvisited candidates | $t$ | $c$ | $w$ | Running total |
|---|---|---|---|---|---|---|
| 1 | $0 \to 1$ | 1: 2.5, 9: 2.6, 4: 3.3 | 2.5 | 0 | **2.5** | 2.5 |
| 2 | $1 \to 9$ | 9: 0.7, 4: 3.3, 3: 3.9 | 0.7 | 0 | **0.7** | 3.2 |
| 3 | $9 \to 4$ | 4: 3.0, 3: 4.3, 8: 9.1 | 3.0 | 0 | **3.0** | 6.2 |
| 4 | $4 \to 3$ | 3: 5.0, 8: 9.9, 2: 11.5 | 5.0 | 0 | **5.0** | 11.2 |
| 5 | $3 \to 8$ | 8: 13.0, 2: 14.1, 7: 14.6 | 8.0 | 5 | **13.0** | 24.2 |
| 6 | $8 \to 7$ | 7: 7.7, 2: 8.4, 6: 23.7 | 2.7 | 5 | **7.7** | 31.9 |
| 7 | $7 \to 2$ | 2: 9.7, 6: 24.7, 5: 27.2 | 4.7 | 5 | **9.7** | 41.6 |
| 8 | $2 \to 6$ | 6: 26.1, 5: 31.2 | 6.1 | 20 | **26.1** | 67.7 |
| 9 | $6 \to 5$ | 5: 28.5 | 8.5 | 20 | **28.5** | 96.2 |
| 10 | $5 \to 0$ | return to depot | 9.7 | 20 | **29.7** | 125.9 |

The key calculations are:

- **Step 1.** From City Hall the candidates are NMMC (2.5 min), the Red Cross (2.6 min) and Maria Reyna (3.3 min). NMMC wins by only 0.1 minute.
- **Steps 2–4.** The tour stays among the undamaged north-eastern sites: the Red Cross (0.7), only about 80 m from NMMC, then Maria Reyna (3.0) and Capitol University Hospital (5.0), the easternmost site. The running total is 11.2 minutes after four of ten legs.
- **Steps 5–7.** With the north-eastern sites done, the crew drives west to the minor-damage sites: West City Central School (13.0), Carmen National High School (7.7) and J. R. Borja (9.7).
- **Steps 8–10.** Only the two flooded riverside sites remain, and every remaining edge carries a 20-minute clearing penalty: Macasandig (26.1), Balulang (28.5) and the return to City Hall (29.7). **These three legs cost 84.3 of the 125.9 minutes.** This is the typical greedy pattern: cheap edges first, expensive edges forced at the end.

Adding the chosen weights gives

$$2.5 + 0.7 + 3.0 + 5.0 + 13.0 + 7.7 + 9.7 + 26.1 + 28.5 + 29.7 = 125.9 \text{ minutes.}$$

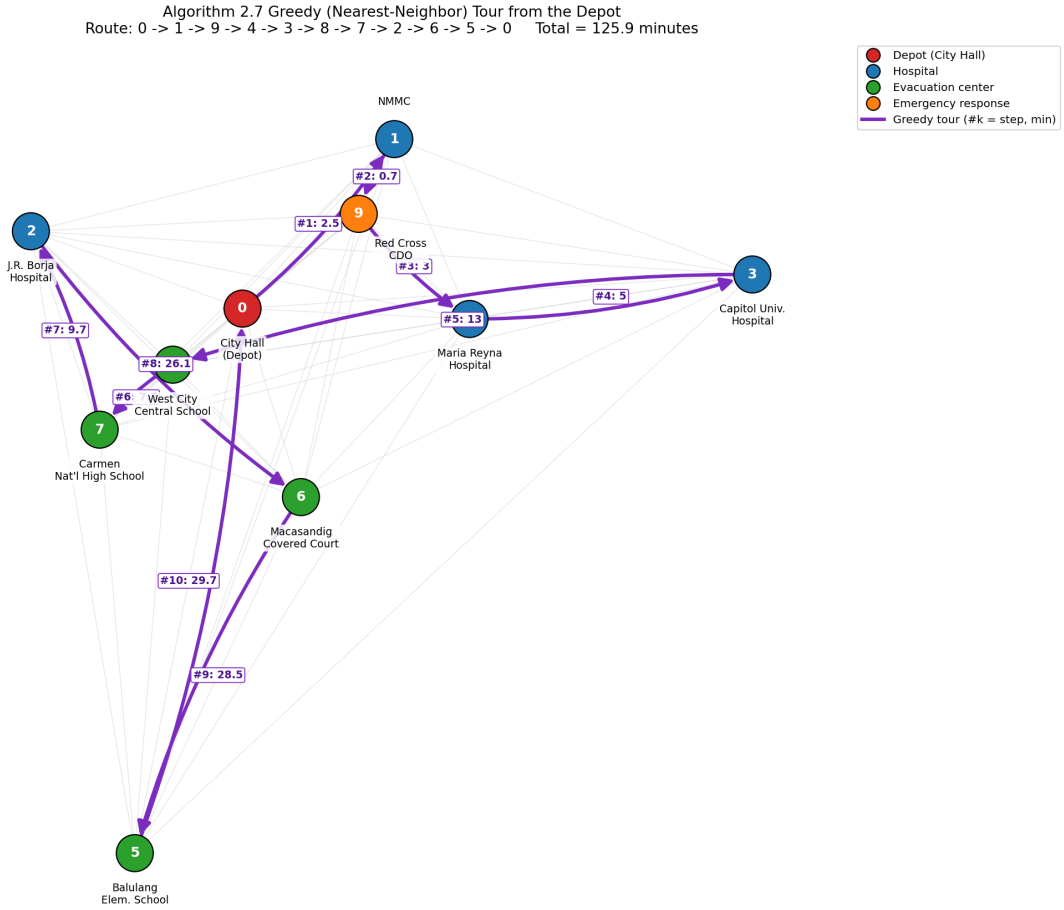



**Figure 3:** The greedy tour on the graph model. Arrows give the direction of travel; labels $\#k$: $w$ give the step number and the edge weight.

# 6 Python implementation

## 6.1 Structure

The implementation uses Python 3.13.7 with OSMnx 2.1.1, NetworkX 3.6.1 [8] and Matplotlib 3.10.7 [9]. It consists of three scripts and a `data/` folder:

| File | Purpose |
|---|---|
| `build_osm_dataset.py` | Downloads the road network, snaps the sites, runs Dijkstra and writes `data/*.csv` |
| `tsp_greedy_cdo.py` | Site list, weight matrix, Algorithm 2.7, verification and the Held–Karp optimality check |
| `tsp_graph_visualization.py` | Draws the graph, the weight matrices and the route maps in `figures/` |
| `data/nodes.csv` | Site coordinates, damage class, snapped OSM node and snap distance |
| `data/travel_time_osm_min.csv` | $10 \times 10$ travel-time matrix in minutes (the main input) |
| `data/distance_osm_km.csv` | $10 \times 10$ road-distance matrix in kilometres |

The scripts are run in this order:

```
pip install osmnx networkx matplotlib
python build_osm_dataset.py
python tsp_greedy_cdo.py
python tsp_graph_visualization.py
```

The data is not embedded in the code: `tsp_greedy_cdo.py` reads the actual travel times from `data/travel_time_osm_min.csv`, and the assumed damage classes are in the `SITES` list at the top of that file. The complete source code is in Appendix A.

## 6.2 Main components

**Input data.** The `SITES` list holds each site's name, category, coordinates and assumed damage class. `load_travel_time()` reads the travel-time CSV, `clearing_time()` applies $c(i,j) = \max(d_i, d_j)$, and `load_weights()` adds them into $W$.

**Listing 1:** Building the weight matrix (`tsp_greedy_cdo.py`).

```
66 def clearing_time():
67     """ASSUMED DATA: clearing an edge costs the worse damage class of its two endpoints."""
68     damage = [s[4] for s in SITES]
69     return [[0 if i == j else max(damage[i], damage[j]) for j in range(N)] for i in range(N)]
70
71
72 def load_weights():
73     """Combined weight matrix W[i][j] = travel + clearing (minutes)."""
74     travel = load_travel_time()
75     clear = clearing_time()
76     return [[round(travel[i][j] + clear[i][j], 1) for j in range(N)] for i in range(N)]
```

**Algorithm 2.7.** The function `greedy_tsp` follows Algorithm 2 line by line. The inner `for` loop is the "find the nearest unvisited city" step; the lines after the `while` loop close the tour.

**Listing 2:** Algorithm 2.7 (`greedy_tsp` in `tsp_greedy_cdo.py`).

```
90 def greedy_tsp(weights, start=0):
91     n = len(weights)
92     visited = [False] * n
93     tour = [start]
94     visited[start] = True
95     total = 0
96     current = start
97
98     while len(tour) < n:
99         # Find the nearest unvisited city (locally optimal choice)
100        next_city = -1
101        best = float("inf")
102        for j in range(n):
103            if not visited[j] and weights[current][j] < best:
104                best = weights[current][j]
105                next_city = j
106        tour.append(next_city)
107        visited[next_city] = True
108        total += best
109        current = next_city
110
111    # Close the loop: return to the starting city
112    total += weights[current][start]
113    tour.append(start)
114    return tour, round(total, 1)
```

**Verification.** `recalculate` independently sums the edge weights along the returned tour, and the report checks that the visited sites are exactly $\{0, \ldots, 9\}$ and that the tour returns to the start.

**Optimality check.** `held_karp` computes the exact optimum by dynamic programming over subsets in $O(n^2 2^n)$ time [5], which is Algorithm 2.2 of the paper. It is not part of Algorithm 2.7; it is used only to measure how far the greedy result is from the best possible tour.

### 6.3 Program output

The output below is the actual run of `python tsp_greedy_cdo.py` on the dataset in this report.

```
================================================================================================
ALGORITHM 2.7 - GREEDY (NEAREST-NEIGHBOR) APPROACH FOR TSP
Application: Post-disaster emergency routing, Cagayan de Oro City (OpenStreetMap roads)
================================================================================================

Tour sequence:
  Step  0: [0] Cagayan de Oro City Hall (DEPOT)
  Step  1: [1] Northern Mindanao Medical Center
  Step  2: [9] Philippine Red Cross - CDO Chapter
  Step  3: [4] Maria Reyna-Xavier Univ. Hospital
  Step  4: [3] Capitol University Hospital
  Step  5: [8] West City Central School (evac)
  Step  6: [7] Carmen National High School (evac)
  Step  7: [2] J. R. Borja Memorial City Hospital
  Step  8: [6] Macasandig Multipurpose Covered Court
  Step  9: [5] Balulang Elementary School (evac)
  Step 10: [0] Cagayan de Oro City Hall (DEPOT)  (return)

Edge-by-edge breakdown (minutes):
  From                                 To                                   travel clear      w
  Cagayan de Oro City Hall (DEPOT)     Northern Mindanao Medical Center        2.5     0    2.5
  Northern Mindanao Medical Center     Philippine Red Cross - CDO Chapter      0.7     0    0.7
  Philippine Red Cross - CDO Chapter   Maria Reyna-Xavier Univ. Hospital       3.0     0    3.0
  Maria Reyna-Xavier Univ. Hospital    Capitol University Hospital             5.0     0    5.0
  Capitol University Hospital          West City Central School (evac)         8.0     5   13.0
  West City Central School (evac)      Carmen National High School (evac)      2.7     5    7.7
  Carmen National High School (evac)   J. R. Borja Memorial City Hospital      4.7     5    9.7
  J. R. Borja Memorial City Hospital   Macasandig Multipurpose Covered Court   6.1    20   26.1
  Macasandig Multipurpose Covered Court Balulang Elementary School (evac)      8.5    20   28.5
  Balulang Elementary School (evac)    Cagayan de Oro City Hall (DEPOT)        9.7    20   29.7

Verification:
  Every node visited exactly once : True
  Returns to starting node        : True
  Reported total time             : 125.9 minutes
  Recomputed total time           : 125.9 minutes

Conclusion:
  Greedy produces an APPROXIMATE (heuristic) solution.
  It does NOT guarantee the global optimum - see paper Theorem 2.7.
================================================================================================

Sensitivity to starting city (greedy from each node):
  start = 0 (Cagayan de Oro City Hall (DEPOT)    ) -> total = 125.9 min
  start = 1 (Northern Mindanao Medical Center    ) -> total = 128.4 min
  start = 2 (J. R. Borja Memorial City Hospital  ) -> total = 124.7 min
  start = 3 (Capitol University Hospital         ) -> total = 128.2 min
  start = 4 (Maria Reyna-Xavier Univ. Hospital   ) -> total = 130.1 min
  start = 5 (Balulang Elementary School (evac)   ) -> total = 121.3 min
  start = 6 (Macasandig Multipurpose Covered Cour) -> total = 124.8 min
  start = 7 (Carmen National High School (evac)  ) -> total = 121.3 min
  start = 8 (West City Central School (evac)     ) -> total = 123.9 min
  start = 9 (Philippine Red Cross - CDO Chapter  ) -> total = 128.8 min

Optimality check (exact Held-Karp DP, paper Algorithm 2.2):
  Optimal tour : 0 -> 2 -> 8 -> 7 -> 5 -> 6 -> 4 -> 3 -> 1 -> 9 -> 0
  Optimal total: 117.0 min   (recomputed 117.0)
  Greedy total : 125.9 min   -> greedy is 7.6% above optimal
```

# 7 Results

### 7.1 Greedy route

The greedy round trip from City Hall is

$$0 \to 1 \to 9 \to 4 \to 3 \to 8 \to 7 \to 2 \to 6 \to 5 \to 0,$$

that is City Hall → NMMC → Red Cross → Maria Reyna → Capitol University Hospital → West City Central School → Carmen National High School → J. R. Borja → Macasandig → Balulang → City Hall. Its total is **125.9 minutes**: 50.9 minutes of driving and 75 minutes of road clearing, over 27.03 km of road. Figure 4 shows the route on the real streets.

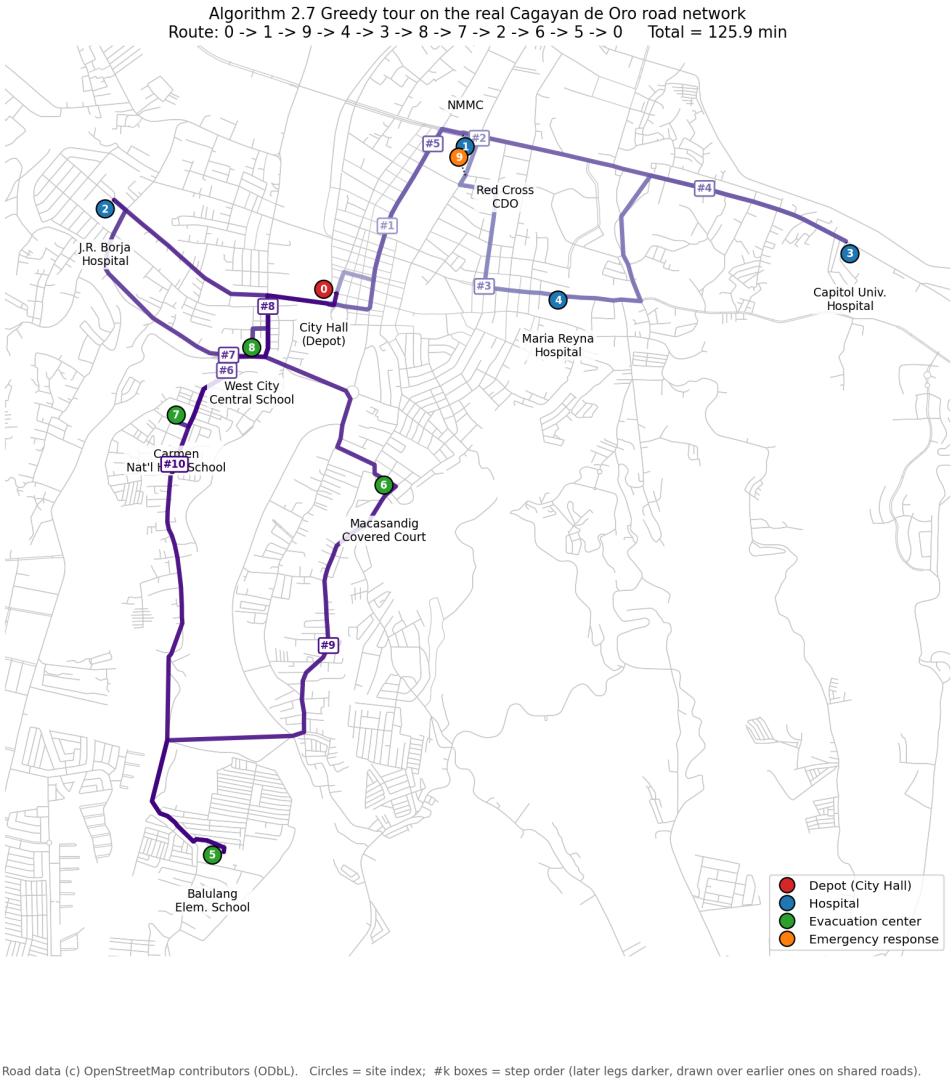



**Figure 4:** The greedy tour on the OpenStreetMap road network of Cagayan de Oro. Circles are sites; $\#k$ boxes give the step order. Road data © OpenStreetMap contributors (ODbL).

## 7.2 Verification

The program output confirms that:

- every one of the ten sites appears exactly once before the return;
- the tour starts and ends at City Hall; and
- the total recomputed from the edge weights equals the reported total, 125.9 minutes. The hand calculation in Section 5 gives the same value.

## 7.3 Comparison with the optimum

Because $n = 10$ is small, the exact optimum can be computed. The Held–Karp dynamic program gives the tour in Table 8 with a total of **117.0 minutes**. A brute-force search over all $9! = 362{,}880$ tours that start and end at City Hall gives the same tour and total, which confirms that it is optimal for this graph.

**Table 8:** Exact optimal tour (Held–Karp dynamic programming), leg by leg.

| Leg | Sites | $t$ | $c$ | $w$ | Running total |
|---|---|---|---|---|---|
| $0 \to 2$ | City Hall → J.R. Borja | 3.4 | 5 | 8.4 | 8.4 |
| $2 \to 8$ | J.R. Borja → West City CS | 3.4 | 5 | 8.4 | 16.8 |
| $8 \to 7$ | West City CS → Carmen NHS | 2.7 | 5 | 7.7 | 24.5 |
| $7 \to 5$ | Carmen NHS → Balulang ES | 7.2 | 20 | 27.2 | 51.7 |
| $5 \to 6$ | Balulang ES → Macasandig CC | 8.5 | 20 | 28.5 | 80.2 |
| $6 \to 4$ | Macasandig CC → Maria Reyna | 4.5 | 20 | 24.5 | 104.7 |
| $4 \to 3$ | Maria Reyna → Capitol Univ. | 5.0 | 0 | 5.0 | 109.7 |
| $3 \to 1$ | Capitol Univ. → NMMC | 4.0 | 0 | 4.0 | 113.7 |
| $1 \to 9$ | NMMC → Red Cross | 0.7 | 0 | 0.7 | 114.4 |
| $9 \to 0$ | Red Cross → City Hall | 2.6 | 0 | 2.6 | 117.0 |

**Table 9:** Greedy versus optimal tour.

| Tour | Total (min) | Driving (min) | Clearing (min) | Distance (km) |
|---|---|---|---|---|
| Greedy (Algorithm 2.7) | 125.9 | 50.9 | 75 | 27.03 |
| Optimal (Held–Karp) | 117.0 | 42.0 | 75 | 22.18 |

The greedy tour is 8.9 minutes, or **7.6%**, longer than the optimum. Both tours visit Balulang and Macasandig back to back, so each pays the 20-minute clearing penalty on exactly three edges and both spend the same 75 minutes clearing roads. (A tour that separated those two sites would pay the penalty on four edges.) The whole gap therefore comes from driving, 50.9 versus 42.0 minutes: the optimal tour visits the riverside sites in the middle of the trip, while greedy postpones them and then has to drive back to City Hall from Balulang, the farthest site, at the end.

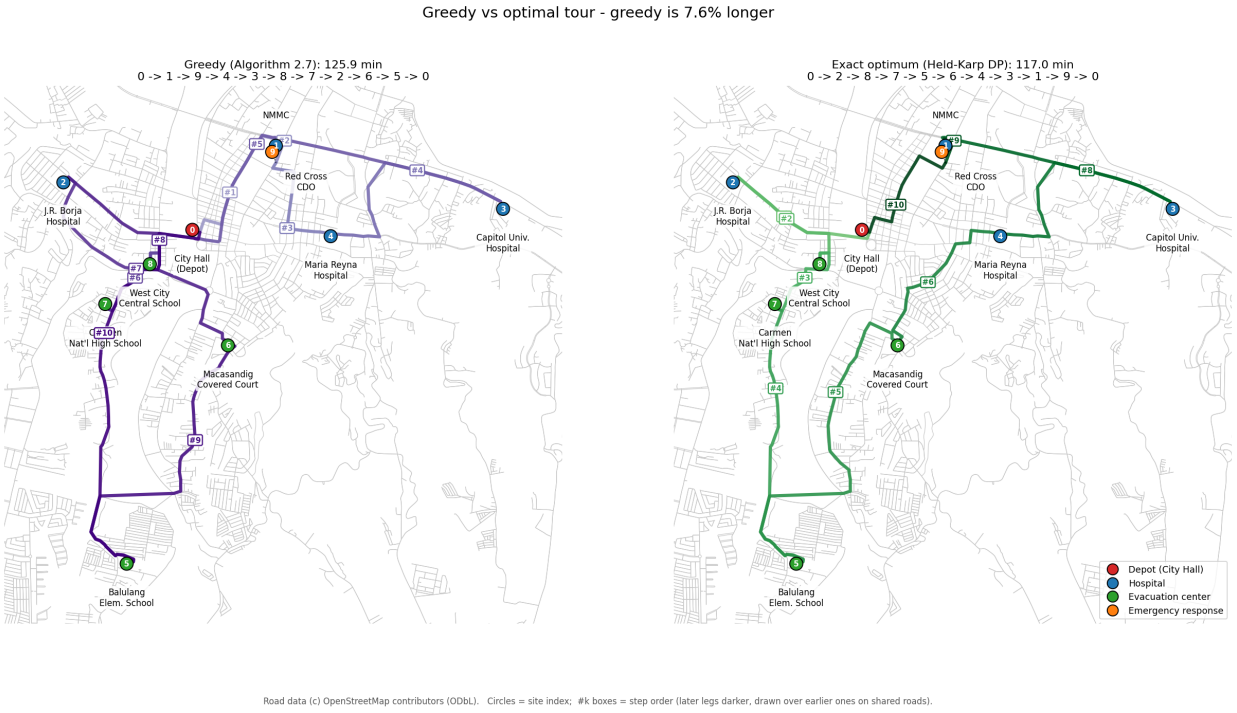



**Figure 5:** Greedy tour (left, 125.9 min) and optimal tour (right, 117.0 min) on the same road network.

### 7.4 Sensitivity to the starting vertex

Running the greedy algorithm from each of the ten sites gives totals between 121.3 and 130.1 minutes (Table 10). None reaches the optimum of 117.0 minutes, and starting from City Hall is not the best choice. This confirms two of the limitations in Section 2: the result depends on the start, and it is not guaranteed to be optimal.

**Table 10:** Greedy result when started from each site. Every closed tour is a cycle, so its total is also a valid depot round trip.

| Start | Site | Greedy tour | Total (min) |
|---|---|---|---|
| 0 | City Hall | $0 \to 1 \to 9 \to 4 \to 3 \to 8 \to 7 \to 2 \to 6 \to 5 \to 0$ | 125.9 |
| 1 | NMMC | $1 \to 9 \to 0 \to 4 \to 3 \to 8 \to 7 \to 2 \to 6 \to 5 \to 1$ | 128.4 |
| 2 | J.R. Borja | $2 \to 0 \to 1 \to 9 \to 4 \to 3 \to 8 \to 7 \to 6 \to 5 \to 2$ | 124.7 |
| 3 | Capitol Univ. | $3 \to 1 \to 9 \to 0 \to 4 \to 8 \to 7 \to 2 \to 6 \to 5 \to 3$ | 128.2 |
| 4 | Maria Reyna | $4 \to 9 \to 1 \to 0 \to 3 \to 8 \to 7 \to 2 \to 6 \to 5 \to 4$ | 130.1 |
| 5 | Balulang ES | $5 \to 7 \to 8 \to 0 \to 1 \to 9 \to 4 \to 3 \to 2 \to 6 \to 5$ | 121.3 |
| 6 | Macasandig CC | $6 \to 0 \to 1 \to 9 \to 4 \to 3 \to 8 \to 7 \to 2 \to 5 \to 6$ | 124.8 |
| 7 | Carmen NHS | $7 \to 8 \to 0 \to 1 \to 9 \to 4 \to 3 \to 2 \to 6 \to 5 \to 7$ | 121.3 |
| 8 | West City CS | $8 \to 0 \to 1 \to 9 \to 4 \to 3 \to 2 \to 7 \to 6 \to 5 \to 8$ | 123.9 |
| 9 | Red Cross | $9 \to 1 \to 0 \to 4 \to 3 \to 8 \to 7 \to 2 \to 6 \to 5 \to 9$ | 128.8 |

## 8 Conclusion

### 8.1 Interpretation

The greedy route meets the objective of the problem in a practical sense: it gives the emergency crew a valid order in which to reach all four hospitals, four evacuation sites and the response hub, each exactly once, and return to City Hall in 125.9 minutes. It is computed in milliseconds and is easy to explain and follow. It is *not* the shortest possible route: the proven optimum is 117.0 minutes, 7.6% faster. We claim optimality only for the Held–Karp result, which is supported by both dynamic programming and brute force; the greedy route is an approximate solution.

### 8.2 Limitations

- Driving speeds and clearing times are assumptions, not measurements, so the absolute minutes are estimates.
- The road network is static. In a real flood some roads would be closed, and travel times would change during the trip.
- The model has no site priorities or time windows, although hospitals may need to be reached first.
- Service time at each site and the walk from the road to each building are ignored.
- The greedy rule postpones the far, flooded sites to the end of the trip, which is exactly where the gap to the optimum comes from.

### 8.3 Possible improvements

- Apply 2-opt local search [7] to the greedy tour to remove its long late legs.
- Run the greedy algorithm from every starting vertex and keep the best tour (at $O(n^3)$ cost).
- Use live road-closure and traffic data instead of assumed speeds and clearing times.
- Add site priorities or time windows to the objective.
- Compare with the metaheuristics in the paper: simulated annealing, genetic algorithms and particle swarm optimization [1].

## References

[1] R. Luna-Garcia, T. M. C. Candano, and R. L. Caga-anan, "Performance analysis of classical algorithms for the traveling salesman problem," *The Mindanawan Journal of Mathematics*, vol. 6, no. 2, pp. 79–111, 2024. https://doi.org/10.62071/tmjm.v6i2.716

[2] OpenStreetMap contributors, "OpenStreetMap data for Cagayan de Oro City," Open Database License (ODbL), accessed 17 October 2026. https://www.openstreetmap.org

[3] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.

[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[5] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962.

[6] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "An analysis of several heuristics for the traveling salesman problem," *SIAM Journal on Computing*, vol. 6, no. 3, pp. 563–581, 1977.

[7] G. A. Croes, "A method for solving traveling-salesman problems," *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958.

[8] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference*, pp. 11–15, 2008.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

# A Source code

## A.1 `tsp_greedy_cdo.py`

```
"""
Performance Innovative Task - Traveling Salesperson Problem (TSP)
Algorithm 2.7: Greedy (Nearest-Neighbor) Approach
Reference: Luna-Garcia, Candano & Caga-anan (2024),
"Performance Analysis of Classical Algorithms for the Traveling Salesman Problem",
The Mindanawan Journal of Mathematics, Vol. 6 (2024), No. 2, pp. 79-111.

Real-life application:
    Post-disaster emergency routing in Cagayan de Oro City.
    An emergency response crew starts at the depot (City Hall), must visit
    every critical emergency site exactly once (hospitals, evacuation
    centers, and an emergency response center), and return to the depot
    while minimizing total round-trip time.

Edge weight definition (directed, because the real road network has one-way streets):
    w(i, j) = travel_time(i, j)  +  clearing_time(i, j)
              (minutes, ACTUAL:      (minutes, ASSUMED:
               shortest path on       disaster scenario
               OpenStreetMap roads)   damage classes)

Run order:
    1. python build_osm_dataset.py      (downloads OSM roads, writes data/*.csv)
    2. python tsp_greedy_cdo.py         (runs Algorithm 2.7 on that dataset)
"""

import csv
import os
```

```
28
29 HERE = os.path.dirname(os.path.abspath(__file__))
30 DATA_DIR = os.path.join(HERE, "data")
31 TRAVEL_TIME_CSV = os.path.join(DATA_DIR, "travel_time_osm_min.csv")
32
33 # ---------------------------------------------------------------------------
34 # 1. NODES (depot = index 0)
35 #    Coordinates: ACTUAL, from OpenStreetMap (Nominatim / Overpass lookups).
36 #    Damage class: ASSUMED scenario - Cagayan River flooding similar to
37 #    Tropical Storm Sendong (2011); riverside sites have the worst access roads.
38 #        0 = clear road, 5 = minor debris, 20 = major obstruction (minutes)
39 # ---------------------------------------------------------------------------
40 SITES = [
41     # (name,                                  category,    lat,      lon,       damage)
42     ("Cagayan de Oro City Hall (DEPOT)",      "depot",     8.47695, 124.64117,  0),
43     ("Northern Mindanao Medical Center",      "hospital",  8.48539, 124.64964,  0),
44     ("J. R. Borja Memorial City Hospital",    "hospital",  8.48171, 124.62809,  5),
45     ("Capitol University Hospital",           "hospital",  8.47905, 124.67268,  0),
46     ("Maria Reyna-Xavier Univ. Hospital",     "hospital",  8.47631, 124.65520,  0),
47     ("Balulang Elementary School (evac)",     "evac",      8.44344, 124.63451, 20),
48     ("Macasandig Multipurpose Covered Court", "evac",      8.46535, 124.64478, 20),
49     ("Carmen National High School (evac)",    "evac",      8.46950, 124.63234,  5),
50     ("West City Central School (evac)",       "evac",      8.47350, 124.63686,  5),
51     ("Philippine Red Cross - CDO Chapter",    "emergency", 8.48477, 124.64926,  0),
52 ]
53 NODES = [s[0] for s in SITES]
54 N = len(SITES)
55
56
57 def load_travel_time():
58     """ACTUAL DATA: OSM shortest-path driving times (minutes), built by build_osm_dataset.py."""
59     if not os.path.exists(TRAVEL_TIME_CSV):
60         raise SystemExit(f"Missing {TRAVEL_TIME_CSV}\nRun:  python build_osm_dataset.py")
61     with open(TRAVEL_TIME_CSV, newline="", encoding="utf-8") as f:
62         rows = list(csv.reader(f))[1:]
63     return [[float(x) for x in row[1:]] for row in rows]
64
65
66 def clearing_time():
67     """ASSUMED DATA: clearing an edge costs the worse damage class of its two endpoints."""
68     damage = [s[4] for s in SITES]
69     return [[0 if i == j else max(damage[i], damage[j]) for j in range(N)] for i in range(N)]
70
71
72 def load_weights():
73     """Combined weight matrix W[i][j] = travel + clearing (minutes)."""
74     travel = load_travel_time()
75     clear = clearing_time()
76     return [[round(travel[i][j] + clear[i][j], 1) for j in range(N)] for i in range(N)]
77
78
79 # ---------------------------------------------------------------------------
80 # 2. ALGORITHM 2.7  --  Greedy (Nearest-Neighbor) Approach for TSP
81 #    Pseudocode (from the paper):
82 #       Input:  cities + pairwise distances
83 #       1. Begin at an arbitrary city
84 #       2. while not all cities visited:
85 #            find nearest unvisited city, travel to it, mark visited
86 #       3. Return to the starting city
87 #       Output: complete tour + total distance
88 #    Time complexity: O(n^2)
89 # ---------------------------------------------------------------------------
90 def greedy_tsp(weights, start=0):
91     n = len(weights)
92     visited = [False] * n
93     tour = [start]
94     visited[start] = True
95     total = 0
96     current = start
97
98     while len(tour) < n:
99         # Find the nearest unvisited city (locally optimal choice)
100        next_city = -1
101        best = float("inf")
102        for j in range(n):
```

```
            if not visited[j] and weights[current][j] < best:
                best = weights[current][j]
                next_city = j
        tour.append(next_city)
        visited[next_city] = True
        total += best
        current = next_city

    # Close the loop: return to the starting city
    total += weights[current][start]
    tour.append(start)
    return tour, round(total, 1)


# ---------------------------------------------------------------------------
# 3. OPTIMALITY CHECK (not part of Algorithm 2.7)
#    Exact optimum via Held-Karp dynamic programming (paper Algorithm 2.2),
#    O(n^2 2^n) - feasible here because n = 10. Used only to measure greedy's gap.
# ---------------------------------------------------------------------------
def held_karp(weights, start=0):
    n = len(weights)
    others = [c for c in range(n) if c != start]
    dp = {(1 << k, k): (weights[start][others[k]], None) for k in range(len(others))}
    for mask in range(1, 1 << len(others)):
        for k in range(len(others)):
            if not mask & (1 << k) or (mask, k) not in dp:
                continue
            cost = dp[(mask, k)][0]
            for nxt in range(len(others)):
                if mask & (1 << nxt):
                    continue
                key = (mask | (1 << nxt), nxt)
                cand = cost + weights[others[k]][others[nxt]]
                if key not in dp or cand < dp[key][0]:
                    dp[key] = (cand, k)
    full = (1 << len(others)) - 1
    last = min(range(len(others)), key=lambda k: dp[(full, k)][0] + weights[others[k]][start])
    total = dp[(full, last)][0] + weights[others[last]][start]
    path, mask, k = [], full, last
    while k is not None:
        path.append(others[k])
        mask, k = mask ^ (1 << k), dp[(mask, k)][1]
    return [start] + path[::-1] + [start], round(total, 1)


# ---------------------------------------------------------------------------
# 4. OUTPUT
# ---------------------------------------------------------------------------
def recalculate(tour, weights):
    """Independent check: sum edge weights along the tour."""
    return round(sum(weights[tour[k]][tour[k + 1]] for k in range(len(tour) - 1)), 1)


def print_report(tour, total, weights):
    travel = load_travel_time()
    clear = clearing_time()
    print("=" * 92)
    print("ALGORITHM 2.7 - GREEDY (NEAREST-NEIGHBOR) APPROACH FOR TSP")
    print("Application: Post-disaster emergency routing, Cagayan de Oro City (OpenStreetMap roads)")
    print("=" * 92)

    print("\nTour sequence:")
    for step, idx in enumerate(tour):
        marker = "  (return)" if step == len(tour) - 1 else ""
        print(f"  Step {step:>2}: [{idx}] {NODES[idx]}{marker}")

    print("\nEdge-by-edge breakdown (minutes):")
    print(f"  {'From':<38} {'To':<38} {'travel':>6} {'clear':>5} {'w':>6}")
    for k in range(len(tour) - 1):
        i, j = tour[k], tour[k + 1]
        print(f"  {NODES[i]:<38} {NODES[j]:<38} {travel[i][j]:>6.1f} {clear[i][j]:>5}
     {weights[i][j]:>6.1f}")

    print("\nVerification:")
    print(f"  Every node visited exactly once : {sorted(tour[:-1]) == list(range(N))}")
```

```
    print(f"  Returns to starting node        : {tour[0] == tour[-1]}")
    print(f"  Reported total time             : {total:.1f} minutes")
    print(f"  Recomputed total time           : {recalculate(tour, weights):.1f} minutes")

    print("\nConclusion:")
    print("  Greedy produces an APPROXIMATE (heuristic) solution.")
    print("  It does NOT guarantee the global optimum - see paper Theorem 2.7.")
    print("=" * 92)


if __name__ == "__main__":
    W = load_weights()
    tour, total = greedy_tsp(W, start=0)
    print_report(tour, total, W)

    # Greedy from every node shows the sensitivity-to-starting-city limitation.
    # Each closed tour is a cycle, so its cost is valid for the depot as well.
    print("\nSensitivity to starting city (greedy from each node):")
    for s in range(N):
        _, cost = greedy_tsp(W, start=s)
        print(f"  start = {s} ({NODES[s][:36]:<36}) -> total = {cost:.1f} min")

    opt_tour, opt_total = held_karp(W, start=0)
    gap = 100 * (total - opt_total) / opt_total
    print("\nOptimality check (exact Held-Karp DP, paper Algorithm 2.2):")
    print(f"  Optimal tour : {' -> '.join(map(str, opt_tour))}")
    print(f"  Optimal total: {opt_total:.1f} min   (recomputed {recalculate(opt_tour, W):.1f})")
    print(f"  Greedy total : {total:.1f} min   -> greedy is {gap:.1f}% above optimal")
```

## A.2 build_osm_dataset.py

```
"""
Builds the ACTUAL travel-time dataset for the TSP from the OpenStreetMap road network.

Steps:
    1. Download the drivable road network around the sites (OSMnx -> Overpass API).
    2. Keep the largest strongly connected component, so every site can reach every other.
    3. Assign a speed to each road segment and compute its travel time
       (length / speed). OSM "maxspeed" tags are used where present;
       otherwise the urban speeds in HWY_SPEEDS (an assumption) are used.
    4. Snap each site to its nearest road intersection.
    5. Run Dijkstra's shortest path between every ordered pair of sites.

Outputs (data/):
    nodes.csv                 site coordinates, snapped OSM node, snap distance
    travel_time_osm_min.csv   n x n driving time matrix (minutes)  <- used by the TSP
    distance_osm_km.csv       n x n road distance along those same routes (km)
    cdo_drive.graphml         cached road network (re-used on later runs)

Road data (c) OpenStreetMap contributors, ODbL.
"""

import csv
import math
import os

import networkx as nx
import osmnx as ox

from tsp_greedy_cdo import DATA_DIR, N, SITES, TRAVEL_TIME_CSV

GRAPH_FILE = os.path.join(DATA_DIR, "cdo_drive.graphml")
BBOX_PAD_DEG = 0.02  # ~2.2 km margin so detours around the sites stay inside the map
OVERPASS_MIRRORS = [
    "https://overpass-api.de/api",
    "https://overpass.private.coffee/api",
    "https://overpass.kumi.systems/api",
    "https://maps.mail.ru/osm/tools/overpass/api",
]

# ASSUMED typical urban speeds (km/h) for Cagayan de Oro traffic, by OSM road class.
HWY_SPEEDS = {
```

```
    "motorway": 60, "motorway_link": 40,
    "trunk": 40, "trunk_link": 30,
    "primary": 35, "primary_link": 25,
    "secondary": 30, "secondary_link": 25,
    "tertiary": 25, "tertiary_link": 20,
    "unclassified": 20, "residential": 20,
    "living_street": 10, "service": 15,
}
FALLBACK_SPEED = 20


def load_graph():
    if os.path.exists(GRAPH_FILE):
        print(f"Using cached road network: {GRAPH_FILE}")
        return ox.load_graphml(GRAPH_FILE)

    lats = [s[2] for s in SITES]
    lons = [s[3] for s in SITES]
    bbox = (min(lons) - BBOX_PAD_DEG, min(lats) - BBOX_PAD_DEG,
            max(lons) + BBOX_PAD_DEG, max(lats) + BBOX_PAD_DEG)
    G = None
    for url in OVERPASS_MIRRORS:
        ox.settings.overpass_url = url
        ox.settings.overpass_rate_limit = url == OVERPASS_MIRRORS[0]
        print(f"Downloading OSM drive network for bbox {bbox} from {url} ...")
        try:
            G = ox.graph_from_bbox(bbox, network_type="drive")
            break
        except Exception as e:  # public servers are often busy; try the next mirror
            print(f"  failed: {str(e).splitlines()[0]}")
    if G is None:
        raise SystemExit("All Overpass servers failed; try again in a few minutes.")
    G = ox.truncate.largest_component(G, strongly=True)
    G = ox.add_edge_speeds(G, hwy_speeds=HWY_SPEEDS, fallback=FALLBACK_SPEED)
    G = ox.add_edge_travel_times(G)
    os.makedirs(DATA_DIR, exist_ok=True)
    ox.save_graphml(G, GRAPH_FILE)
    return G


def haversine_m(lat1, lon1, lat2, lon2):
    r = 6371000.0
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp, dl = p2 - p1, math.radians(lon2 - lon1)
    a = math.sin(dp / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl / 2) ** 2
    return 2 * r * math.asin(math.sqrt(a))


def snap_sites(G):
    coords = [(n, d["y"], d["x"]) for n, d in G.nodes(data=True)]
    snapped = []
    for _, _, lat, lon, _ in SITES:
        node, y, x = min(coords, key=lambda c: haversine_m(lat, lon, c[1], c[2]))
        snapped.append((node, haversine_m(lat, lon, y, x)))
    return snapped


def fastest_edge(G, u, v):
    return min(G[u][v].values(), key=lambda d: d["travel_time"])


def route_length_m(G, path):
    return sum(fastest_edge(G, u, v)["length"] for u, v in zip(path[:-1], path[1:]))


def write_matrix(path, matrix, fmt):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["from\\to"] + list(range(N)))
        for i, row in enumerate(matrix):
            w.writerow([i] + [fmt.format(x) for x in row])


def main():
    G = load_graph()
```

```
    print(f"Road network: {G.number_of_nodes()} intersections, {G.number_of_edges()} directed road segments")

    snapped = snap_sites(G)
    osm_nodes = [s[0] for s in snapped]

    minutes = [[0.0] * N for _ in range(N)]
    km = [[0.0] * N for _ in range(N)]
    for i in range(N):
        seconds, paths = nx.single_source_dijkstra(G, osm_nodes[i], weight="travel_time")
        for j in range(N):
            if i != j:
                minutes[i][j] = round(seconds[osm_nodes[j]] / 60, 1)
                km[i][j] = round(route_length_m(G, paths[osm_nodes[j]]) / 1000, 2)

    os.makedirs(DATA_DIR, exist_ok=True)
    with open(os.path.join(DATA_DIR, "nodes.csv"), "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["index", "name", "category", "lat", "lon", "assumed_damage_min",
                    "osm_node", "snap_distance_m"])
        for i, (name, cat, lat, lon, dmg) in enumerate(SITES):
            w.writerow([i, name, cat, lat, lon, dmg, osm_nodes[i], round(snapped[i][1])])
    write_matrix(TRAVEL_TIME_CSV, minutes, "{:.1f}")
    write_matrix(os.path.join(DATA_DIR, "distance_osm_km.csv"), km, "{:.2f}")

    print("\nSite -> nearest road intersection:")
    for i, (node, dist) in enumerate(snapped):
        print(f"  [{i}] {SITES[i][0]:<40} OSM node {node:<12} {dist:6.0f} m away")

    diffs = [abs(minutes[i][j] - minutes[j][i]) for i in range(N) for j in range(i + 1, N)]
    print(f"\nOne-way streets: {sum(d > 0 for d in diffs)} of {len(diffs)} site pairs have "
          f"different times each direction (max difference {max(diffs):.1f} min)")
    print(f"Saved dataset to {DATA_DIR}")


if __name__ == "__main__":
    main()
```

## A.3 tsp_graph_visualization.py

```
"""
Visualizations for the Algorithm 2.7 (Greedy) TSP application, built on the
OpenStreetMap dataset produced by build_osm_dataset.py.

Figures written to ./figures:
    1_weighted_graph.png   complete directed weighted graph (vertices at real coordinates)
    2_greedy_tour.png      greedy tour highlighted on the graph, in visit order
    3_weight_matrices.png  travel (actual) + clearing (assumed) = combined weight matrix W
    4_greedy_route_map.png greedy tour drawn on the real Cagayan de Oro road network
    5_greedy_vs_optimal_map.png  greedy tour vs exact optimum (Held-Karp) on the road map
"""

import csv
import math
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import networkx as nx
import osmnx as ox
from matplotlib.lines import Line2D

from build_osm_dataset import GRAPH_FILE
from tsp_greedy_cdo import (DATA_DIR, HERE, N, SITES, clearing_time, greedy_tsp,
                            held_karp, load_travel_time, load_weights)

OUT_DIR = os.path.join(HERE, "figures")
os.makedirs(OUT_DIR, exist_ok=True)

SHORT = [
```

```
    "City Hall\n(Depot)",
    "NMMC",
    "J.R. Borja\nHospital",
    "Capitol Univ.\nHospital",
    "Maria Reyna\nHospital",
    "Balulang\nElem. School",
    "Macasandig\nCovered Court",
    "Carmen\nNat'l High School",
    "West City\nCentral School",
    "Red Cross\nCDO",
]
COLORS = {"depot": "#d62728", "hospital": "#1f77b4", "evac": "#2ca02c", "emergency": "#ff7f0e"}
LEGEND_LABELS = {"depot": "Depot (City Hall)", "hospital": "Hospital",
                 "evac": "Evacuation center", "emergency": "Emergency response"}
CATEGORY = [s[1] for s in SITES]

# Label offsets in points (dx, dy) so labels of nearby sites do not collide.
LABEL_OFFSET = {i: (0, -24) for i in range(N)}
LABEL_OFFSET.update({1: (0, 26), 9: (34, -20)})
LABEL_VA = {1: "bottom"}

# Graph-figure positions: real coordinates in km from the depot. NMMC and Red Cross
# are only ~80 m apart, so they are nudged apart slightly to keep both visible.
LAT0, LON0 = SITES[0][2], SITES[0][3]
POS = {i: ((s[3] - LON0) * 111.32 * math.cos(math.radians(LAT0)), (s[2] - LAT0) * 110.57)
       for i, s in enumerate(SITES)}
POS[1] = (POS[1][0] + 0.10, POS[1][1] + 0.22)
POS[9] = (POS[9][0] - 0.10, POS[9][1] - 0.22)


def node_legend():
    return [Line2D([0], [0], marker="o", color="w", markerfacecolor=COLORS[c], markeredgecolor="black",
                   markersize=11, label=LEGEND_LABELS[c]) for c in COLORS]


def label_sites(ax, xy):
    for i in range(N):
        ax.annotate(SHORT[i], xy[i], xytext=LABEL_OFFSET[i], textcoords="offset points",
                    ha="center", va=LABEL_VA.get(i, "top"), fontsize=8.5, zorder=6,
                    bbox=dict(boxstyle="round,pad=0.2", fc="white", ec="none", alpha=0.85))


def draw_graph_nodes(ax, G):
    nx.draw_networkx_nodes(G, POS, ax=ax, node_size=900, edgecolors="black",
                           node_color=[COLORS[CATEGORY[i]] for i in G.nodes])
    nx.draw_networkx_labels(G, POS, ax=ax, labels={i: str(i) for i in G.nodes},
                            font_color="white", font_weight="bold", font_size=11)
    label_sites(ax, POS)


def finish_graph_axes(ax, fig, handles, path):
    ax.legend(handles=handles, loc="upper left", bbox_to_anchor=(1.0, 1.0), fontsize=9)
    ax.set_aspect("equal")
    ax.margins(0.08)
    ax.axis("off")
    fig.tight_layout()
    fig.savefig(path, dpi=160)
    plt.close(fig)
    return path


def fig_weighted_graph(W):
    G = nx.Graph()
    G.add_nodes_from(range(N))
    labels = {}
    for i in range(N):
        for j in range(i + 1, N):
            G.add_edge(i, j)
            labels[(i, j)] = f"{W[i][j]:g}" if W[i][j] == W[j][i] else f"{W[i][j]:g}/{W[j][i]:g}"
    fig, ax = plt.subplots(figsize=(14, 11))
    nx.draw_networkx_edges(G, POS, ax=ax, edge_color="#9a9a9a", width=0.8, alpha=0.6)
    nx.draw_networkx_edge_labels(G, POS, ax=ax, edge_labels=labels, font_size=6.5,
                                 bbox=dict(boxstyle="round,pad=0.1", fc="#fffbe6", ec="none", alpha=0.9))
    draw_graph_nodes(ax, G)
    ax.set_title("Weighted Graph Model - Post-Disaster Emergency Routing, Cagayan de Oro\n"
```

```
                "10 vertices at real coordinates; w(i,j) = OSM travel time + assumed clearing time (min)\n"
                "One-way streets make it directed: label 'a/b' = w(i->j)/w(j->i) for i<j", fontsize=11)
    return finish_graph_axes(ax, fig, node_legend(), os.path.join(OUT_DIR, "1_weighted_graph.png"))


def fig_greedy_tour(W, tour, total):
    G = nx.complete_graph(N)
    fig, ax = plt.subplots(figsize=(14, 11))
    nx.draw_networkx_edges(G, POS, ax=ax, edge_color="#cccccc", width=0.6, alpha=0.5)
    tour_edges = list(zip(tour[:-1], tour[1:]))
    D = nx.DiGraph(tour_edges)
    nx.draw_networkx_edges(D, POS, ax=ax, edgelist=tour_edges, edge_color="#7b2cbf", width=2.8,
                           arrows=True, arrowstyle="-|>", arrowsize=22, node_size=900,
                           connectionstyle="arc3,rad=0.08")
    for k, (u, v) in enumerate(tour_edges, start=1):
        (x1, y1), (x2, y2) = POS[u], POS[v]
        ax.text((x1 + x2) / 2, (y1 + y2) / 2, f"#{k}: {W[u][v]:g}", fontsize=8.5, ha="center",
                va="center", color="#4a148c", fontweight="bold", zorder=5,
                bbox=dict(boxstyle="round,pad=0.2", fc="white", ec="#7b2cbf", lw=0.8))
    draw_graph_nodes(ax, G)
    ax.set_title("Algorithm 2.7 Greedy (Nearest-Neighbor) Tour from the Depot\n"
                 f"Route: {' -> '.join(map(str, tour))}     Total = {total:.1f} minutes", fontsize=12)
    handles = node_legend() + [Line2D([0], [0], color="#7b2cbf", lw=2.8, label="Greedy tour (#k = step, min)")]
    return finish_graph_axes(ax, fig, handles, os.path.join(OUT_DIR, "2_greedy_tour.png"))


def fig_weight_matrices(W):
    panels = [
        (load_travel_time(), "Travel time - ACTUAL\n(OpenStreetMap shortest paths)", "Blues"),
        (clearing_time(), "Clearing time - ASSUMED\n(damage class scenario)", "Oranges"),
        (W, "Combined weight W = travel + clearing\n(used by Algorithm 2.7)", "YlOrRd"),
    ]
    fig, axes = plt.subplots(1, 3, figsize=(24, 8.5))
    for ax, (M, title, cmap) in zip(axes, panels):
        vmax = max(max(r) for r in M) or 1
        im = ax.imshow(M, cmap=cmap, vmin=0, vmax=vmax)
        ax.set_xticks(range(N), labels=[str(i) for i in range(N)])
        ax.set_yticks(range(N), labels=[f"{i} {SHORT[i].replace(chr(10), ' ')}" for i in range(N)], fontsize=8.5)
        ax.set_xlabel("to"), ax.set_ylabel("from")
        for i in range(N):
            for j in range(N):
                ax.text(j, i, f"{M[i][j]:g}", ha="center", va="center", fontsize=8.5,
                        color="white" if M[i][j] > vmax * 0.6 else "black")
        ax.set_title(title, fontsize=12)
        fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04, label="minutes")
    fig.suptitle("Weight matrices (minutes), Cagayan de Oro emergency sites", fontsize=14)
    fig.tight_layout()
    path = os.path.join(OUT_DIR, "3_weight_matrices.png")
    fig.savefig(path, dpi=150)
    plt.close(fig)
    return path


# ---------------------------------------------------------------------------- road maps
def load_osm_nodes():
    with open(os.path.join(DATA_DIR, "nodes.csv"), newline="", encoding="utf-8") as f:
        return [int(r["osm_node"]) for r in csv.DictReader(f)]


def draw_road_map(ax, R, osm_nodes, tour, title, cmap_name):
    ox.plot_graph(R, ax=ax, node_size=0, edge_color="#c8c8c8", edge_linewidth=0.6,
                  bgcolor="white", show=False, close=False)
    cmap = plt.get_cmap(cmap_name)
    legs = list(zip(tour[:-1], tour[1:]))
    for k, (u, v) in enumerate(legs, start=1):
        path = nx.shortest_path(R, osm_nodes[u], osm_nodes[v], weight="travel_time")
        xs = [R.nodes[n]["x"] for n in path]
        ys = [R.nodes[n]["y"] for n in path]
        color = cmap(0.5 + 0.5 * (k - 1) / (len(legs) - 1))
        ax.plot(xs, ys, color=color, lw=3.2, alpha=0.9, solid_capstyle="round", zorder=3)
        mid = len(path) // 2
        ax.text(xs[mid], ys[mid], f"#{k}", fontsize=7.5, fontweight="bold", color=color, ha="center",
```

```
                va="center", zorder=7, bbox=dict(boxstyle="round,pad=0.2", fc="white", ec=color, lw=1.2))
    lons = [s[3] for s in SITES]
    lats = [s[2] for s in SITES]
    for i in range(N):  # dashed connector from the building to its snapped road node
        ax.plot([lons[i], R.nodes[osm_nodes[i]]["x"]], [lats[i], R.nodes[osm_nodes[i]]["y"]],
                ls=":", color="black", lw=1, zorder=4)
    ax.scatter(lons, lats, s=170, c=[COLORS[c] for c in CATEGORY], edgecolors="black", zorder=8)
    for i in range(N):
        ax.text(lons[i], lats[i], str(i), fontsize=7.5, fontweight="bold", color="white",
                ha="center", va="center", zorder=9)
    label_sites(ax, list(zip(lons, lats)))
    pad = 0.006
    ax.set_xlim(min(lons) - pad, max(lons) + pad)
    ax.set_ylim(min(lats) - pad, max(lats) + pad)
    ax.set_title(title, fontsize=11.5)


def fig_route_maps(W, greedy, greedy_total, opt, opt_total):
    R = ox.load_graphml(GRAPH_FILE)
    osm_nodes = load_osm_nodes()
    credit = ("Road data (c) OpenStreetMap contributors (ODbL).   Circles = site index;  "
              "#k boxes = step order (later legs darker, drawn over earlier ones on shared roads).")
    handles = node_legend()

    fig, ax = plt.subplots(figsize=(13, 12))
    draw_road_map(ax, R, osm_nodes, greedy,
                  "Algorithm 2.7 Greedy tour on the real Cagayan de Oro road network\n"
                  f"Route: {' -> '.join(map(str, greedy))}     Total = {greedy_total:.1f} min", "Purples")
    ax.legend(handles=handles, loc="lower right", fontsize=9)
    fig.text(0.5, 0.01, credit, ha="center", fontsize=8.5, color="#555555")
    p1 = os.path.join(OUT_DIR, "4_greedy_route_map.png")
    fig.savefig(p1, dpi=160, bbox_inches="tight", facecolor="white")
    plt.close(fig)

    gap = 100 * (greedy_total - opt_total) / opt_total
    fig, axes = plt.subplots(1, 2, figsize=(22, 10))
    draw_road_map(axes[0], R, osm_nodes, greedy,
                  f"Greedy (Algorithm 2.7): {greedy_total:.1f} min\n{' -> '.join(map(str, greedy))}",
     "Purples")
    draw_road_map(axes[1], R, osm_nodes, opt,
                  f"Exact optimum (Held-Karp DP): {opt_total:.1f} min\n{' -> '.join(map(str, opt))}",
     "Greens")
    axes[1].legend(handles=handles, loc="lower right", fontsize=9)
    fig.suptitle(f"Greedy vs optimal tour - greedy is {gap:.1f}% longer", fontsize=15)
    fig.text(0.5, 0.01, credit, ha="center", fontsize=8.5, color="#555555")
    p2 = os.path.join(OUT_DIR, "5_greedy_vs_optimal_map.png")
    fig.savefig(p2, dpi=140, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    return p1, p2


if __name__ == "__main__":
    W = load_weights()
    tour, total = greedy_tsp(W, start=0)
    opt, opt_total = held_karp(W, start=0)
    paths = [fig_weighted_graph(W), fig_greedy_tour(W, tour, total), fig_weight_matrices(W),
             *fig_route_maps(W, tour, total, opt, opt_total)]
    for p in paths:
        print("Saved:", p)
```

# B Road distances

**Table 11:** Road distance in kilometres along the same fastest paths.

| from \ to | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | – | 1.59 | 1.74 | 4.22 | 1.76 | 5.00 | 1.65 | 1.86 | 0.89 | 1.55 |
| **1** | 1.82 | – | 3.35 | 2.63 | 1.75 | 6.82 | 3.00 | 3.70 | 2.73 | 0.37 |
| **2** | 1.75 | 3.10 | – | 5.73 | 3.17 | 5.40 | 3.04 | 2.25 | 1.62 | 2.94 |
| **3** | 4.22 | 2.67 | 5.74 | – | 2.84 | 9.21 | 5.40 | 6.10 | 5.13 | 2.79 |
| **4** | 1.76 | 1.82 | 3.18 | 2.84 | – | 6.58 | 2.29 | 3.64 | 2.67 | 1.48 |
| **5** | 5.00 | 6.57 | 5.39 | 9.20 | 6.40 | – | 4.56 | 3.39 | 4.43 | 6.53 |
| **6** | 1.64 | 2.79 | 3.29 | 4.98 | 2.15 | 4.03 | – | 2.53 | 1.95 | 2.37 |
| **7** | 1.86 | 3.45 | 2.25 | 6.08 | 3.62 | 3.39 | 2.32 | – | 1.28 | 3.41 |
| **8** | 0.90 | 2.50 | 1.62 | 5.13 | 2.67 | 4.45 | 1.76 | 1.30 | – | 2.46 |
| **9** | 1.54 | 0.43 | 2.94 | 2.78 | 1.48 | 6.53 | 2.37 | 3.42 | 2.45 | – |